Reinforcement Learning Experiment Report

Peixuan Wang

***Abstract*— This report is a submission to the required assignment of the reinforcement learning module, which contains the necessary contents about the experiments specified in the information document in Canvas.**

I. INTRODUCTION

This report attempts to give a comprehensive study of main algorithms in Reinforcement Learning (RL) by contrasting and comparing their performance on two Markov Decision Process (MDP) instances with distinct features. The aim of this exercise is to use learned machine learning techniques to enable an agent to move more directly in the world.

For this, the report will focus on two given environments:

- **Blackjack (Blackjack-v1):** A stochastic and discrete game of cards, the actions of the agent stochastically affect future rewards.
- **CartPole (CartPole-v1):** A physics simulation balancing problem, this environment features a continuous state space. The application of dynamic programming methods requires discretization of this state space.

Both of these MDPs also differ substantially with regard to their state spaces, action spaces, and reward structures. For instance, Blackjack is by nature discrete and stochastic, while CartPole is continuous and needs a secondary discretization step. By testing and applying varied RL algorithms within these two varying environments, the present report strives to uncover the relative strength, vulnerability, and sensitivity to environmental characteristics of varied algorithms. The contrast will be on how convergence, policy optimisation, outcomes, and computational complexity vary between model-based dynamic programming and model-free reinforcement learning.

II. HYPOTHESES

In this paper, I will analyze the performance of model-based and model-free reinforcement learning algorithms in two environments. Our hypotheses, based on the theoretical properties of the algorithms and the environments, are as follows:

1. Model-Based vs. Model-Free Superiority: I anticipate that model-based methods, Value Iteration (VI) and Policy Iteration (PI), will acquire more efficient policies and do so with far greater computational efficiency than the model-free SARSA algorithm. This is due to the reality that VI and PI leverage an explicit environment model for planning in deterministic environments, whereas SARSA must settle for sample-inefficient trial-and-error discovery.

2. Convergence Rate of VI and PI: I hope that Policy Iteration will converge faster than Value Iteration. PI specifically searches for a good policy, and in games with a relatively small set of helpful policies like Blackjack, this policy search space will typically be less extensive than the state-value space that VI has to scan exhaustively.

3. Discretization Effect: For the CartPole environment, I anticipate the performance to be very sensitive to the discretization method of the state. A "Rough" discretization will be too rough to capture significant state information and will lead to a poor policy. But a "Precise" discretization will create an immensely large state space (the "curse of dimensionality"), which will hinder the learning procedure, especially for SARSA. I anticipate that a "Medium" discretization level will provide the optimum performance by offering the best compromise between state representation accuracy and learnability.

4. Optimal Exploration Strategy: I expect the optimal exploration strategy for SARSA to be dependent on the environment. For a goal-directed problem like Blackjack, an exponentially decaying epsilon would do well, as it allows the agent to settle onto an optimal policy. But for a dynamic balancing problem like CartPole, a carefully tuned fixed epsilon would possibly perform better, as the need for continuous exploration and tuning never truly dissipates.

III. EXPERIMENTS ON BLACK JACK ENVIRONMENT

A. Environment Introduction

The Blackjack environment is a simplified version of the well-known card game. The objective is to obtain a card sum as close to 21 as possible without exceeding it, a condition known as "busting". The agent plays against a dealer who uses a fixed policy. An agent's state is defined by a tuple containing the player's current sum, the value of the dealer's single visible card, and whether the player holds a usable ace. At each step, the agent has two discrete actions: "hit" (take another card) or "stick" (end the turn). Upon the game's conclusion, the agent receives a reward of +1 for winning, -1 for losing, and 0 for a draw.

This environment is significant for the report because it serves as a classic example of a discrete and stochastic Markov Decision Process (MDP). Its discrete state and action spaces make it an ideal testbed for tabular reinforcement learning methods like Value Iteration, Policy Iteration, and SARSA. Furthermore, the stochastic nature, which arises from the random drawing of cards, introduces uncertainty into the state transitions. This allows for a direct analysis of how these algorithms perform when future outcomes are probabilistic rather than guaranteed. Its properties provide

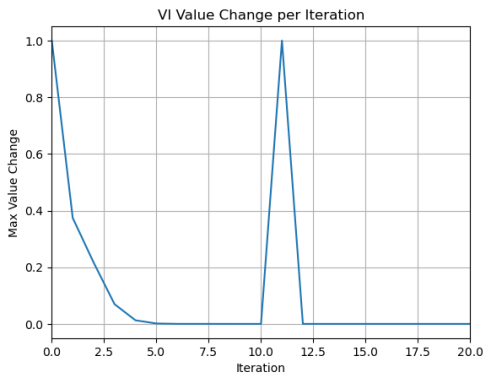

Fig. 1. VI Value Change per Iteration

a fundamental baseline for comparison against the deterministic environment of CartPole, helping to reveal how algorithmic behavior is influenced by the environment's characteristics.

B. Experiments on Value Iteration

I applied value iteration algorithm to learn policies, playing with black jack environment. VI algorithm, explained in simplest words, is to evaluation values of agent reaching each state using bellman equation, and then draw the optimal policies based on that.

1) Convergence: The plot I drew showing the convergence of VI algorithm is shown in Fig.1. The Value Change implies the maximum value change among all value differences between two adjacent iterations, so it can to some extent show how little the changes become as iterations pass.

We can see that values of states experienced a dramatic oscillation around 11-th iteration and them converged at about 12-th iteration.

2) Policy & Performance: I evaluated the performance of the policies I derived from this trial by testing it back in the Blackjack environment and record the rewards it had, then I take the mean and standard deviation of it.

I did 100,000 rounds and get a mean of -0.04116 and a std of 0.95106, which is still very high considering how many repeats I did. I think both of these two numbers make sense. The expectation of this game is negative, which means that the dealers always win, that is exactly how casinos make profits and the variation of rewards among games is incredibly high, because only this kind of games is thrilling enough to attract gamblers.

The policy heatmap in Fig.2. can directly illustrate the policies I got. It shows the 29 states (including "Black Jack") that players might have and 10 states the dealer might have.

The "Hard" states on the upper half are states when player doesn't have any Ace in hand. And "Soft" states are situations when player does have Ace. "Black Jack" means player initially has the 21 points with only the first two cards, which almost suggests a solid win.

This heat map is very practical, it can be noticed player has two significant peaks of values (based on color) and a minor peak.

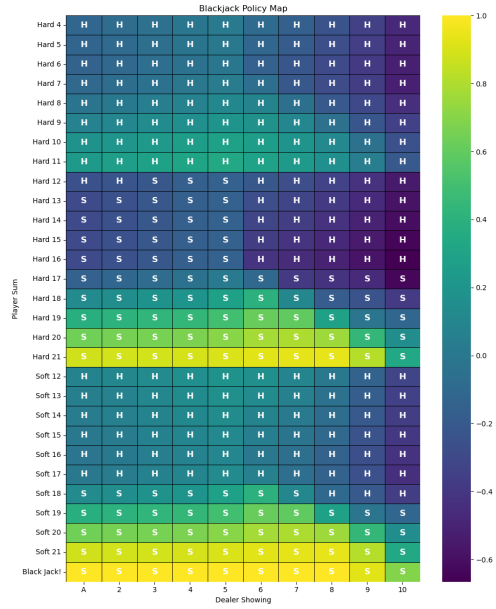

Fig. 2. Blackjack Policy Heatmap(VI)

The minor peak is around "Hard 11", I believe that's because there are many cards standing for 10 points, including 10 and all "face cards", therefore 10 is the easiest number to get. Thus when player has already had 10 points, the probability they'll reach 21 is remarkably high and zero chance they'll exceed it in the next draw. The best policy now is clearly "hit".

The second peak is around "Hard 21", when the player already has 21 points, it's reasonable to stick because any card would explode what the players has now. But the win is not guaranteed, while the dealer's score approaching 10. When it happens, player's value goes down, because the dealer is facing a similar situation the player's facing when at "Hard 11", it's likely they'll end up tie.

The third peak is similar. But it's worth noticing that when player's in a "Hard" states, 17 and some states of 16 points are high enough to stick, because the next draw might lead to player's explosion. However, when getting same scores in "Soft" states, the policy is always hit. That's because the Ace in hand has a flexible value, which can either be 1 or 11, which means the risk of explosion can be lowered by changing Ace from 11 to 1.

C. Experiments on Policy Iteration

The major difference I think might be concerning is that PI basically is searching the policies space instead of totally basing on values of states.

1) Convergence: The convergence of Policy Iteration is plotted in Fig.3. It goes up a little first, I guess that's affected by the initial policies.

The PI algorithms converged at around 5-th iteration, which is a lot faster than VI. I think that's because the policy space in this envornment is relatively small, making the search much easier.

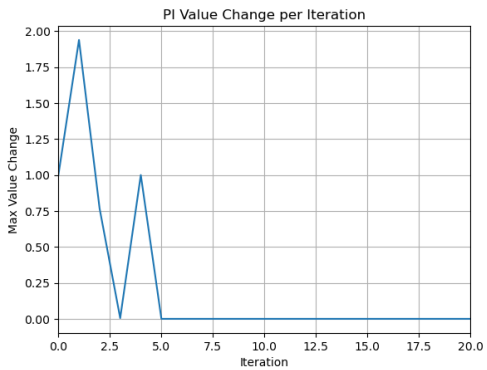

Fig. 3. PI Value Change per Iteration

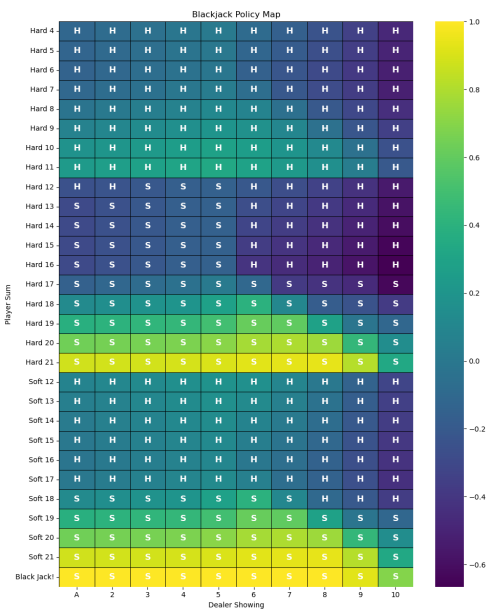

Fig. 4. Blackjack Policy Heatmap(PI)

2) Policy & Performance: The expectation of rewards is -0.04471 and the std is 0.95020. The number of trials I did is also 100,000. Though the mean reward seems a bit worse compared to previous number I had of VI, I'm inclined to not to regard this as a significant difference because of the magnitude of std. The present inferior might be just illusion caused by randomness. I'd love to do more experiments to lower the std but my local computer's not able to handle this workload. Deriving the expectation through probabilistic model might also be a nice idea, except it's not the focus of this course.

The policy heatmap shows the policies provided by these two model-based method are identical, again supporting their similarity.

D. Experiments on SARSA

SARSA is a model-free algorithm mainly focusing on deriving a action value function $Q(s, a)$, which can assess the value of taking a certain action under certain state.

It's required to explore different strategies trading off

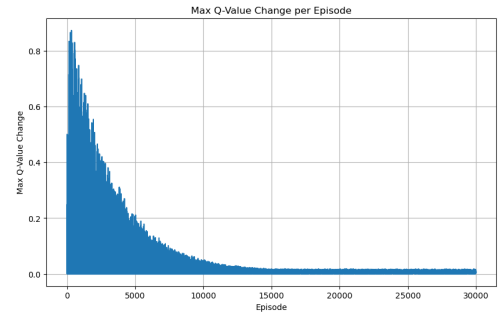

Fig. 5. SARSA-Decay Q Change per Iteration

Exploration & Exploitation. So I select two major types of trategies, testing a decaying ϵ or a fixed one.

1) SARSA using Decaying Epsilon: The ϵ is referred to as "exploration rate", which is the probability of the algorithm seeks to explore broader space instead of focusing on exploitation of present policies.

Firstly, about the convergence, the max Q change is plotted in Fig.5, which exhibits a unique characteristic. On the scale of tens of thousands of episodes, its maximum Q-value change shows drastic, high-frequency oscillations. Due to the density of data points, the plot visually forms an almost "solid" area, which trends slowly toward zero. In stark contrast, the plots for Value Iteration (VI) and Policy Iteration (PI) are simple and clear; they converge rapidly within a very small number of iterations (typically fewer than 20), resulting in a sparse line graph composed of only a few data points.

The root of this significant difference lies in their fundamental algorithmic nature. VI and PI are model-based dynamic programming methods that operate with a complete "map" of the environment (i.e., known state transition probabilities) beforehand. Their "iteration" is a deterministic, global planning calculation based on this map, making the process efficient and stable. SARSA, on the other hand, is a model-free reinforcement learning method that must learn like an explorer through thousands of trial-and-error interactions. Each drastic oscillation on its plot records the maximum "surprise" encountered during an episode as a result of random exploration. Therefore, the sparse plots of VI and PI are a reflection of deterministic planning, while SARSA's "solid," oscillating plot is a vivid depiction of the stochastic exploratory learning process.

The moving average shown in Fig.6 also shows forming of a platform, suggesting convergence. MA plot can be considered as a form of **learning curve** for SARSA.

The Cumulative rewards suggests that the absolute value of marginal reward is decreasing.

Mean Rewards: -0.06065; Std: 0.95180

The performance of SARSA is not as good as what I had with VI and PI. Combined it with Fig.8, there are several dumb policies that is clearly not reasonable. Overall speaking, SARSA's performance is not satisfying. I believe the reason is the intrinsic differences discussed above, SARSA don't have a perception about the whole space.

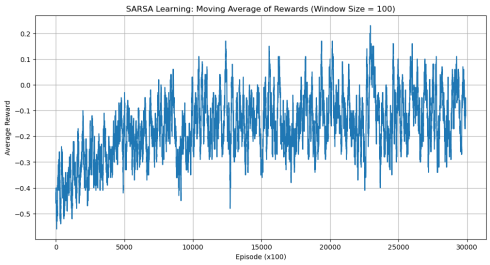

Fig. 6. SARSA-Decay MA Rewards over Iteration

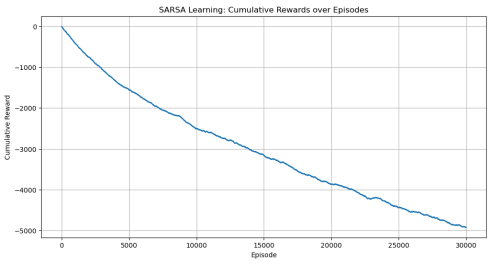

Fig. 7. SARSA-Decay Cummulative Rewards over Iteration

2) SARSA using Fixed Epsilon: If the previous SARSA strategy using a decaying exploration rate shows a dynamic process of algorithms switching from exploration to exploitation, fixed ϵ strategy shows a more static process, making the algorithms to maintain a constant rate of exploration.

I tried 9 ϵ value from 0.1 to 0.9 with step size 0.1. And the outcomes are:

Epsilon=0.1; Mean Reward: -0.07552; Std: 0.94775
Epsilon=0.2; Mean Reward: -0.06358; Std: 0.95021
Epsilon=0.3; Mean Reward: -0.06834; Std: 0.95371
Epsilon=0.4; Mean Reward: -0.06504; Std: 0.95460
Epsilon=0.5; Mean Reward: -0.07723; Std: 0.95722

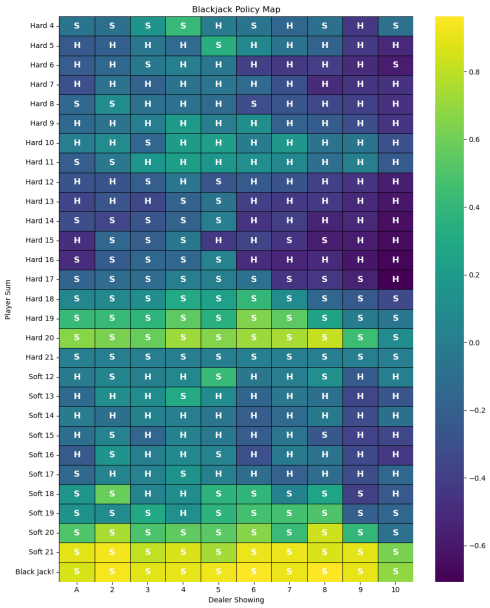

Fig. 8. Blackjack Policy Heatmap(SARSA-Decay)

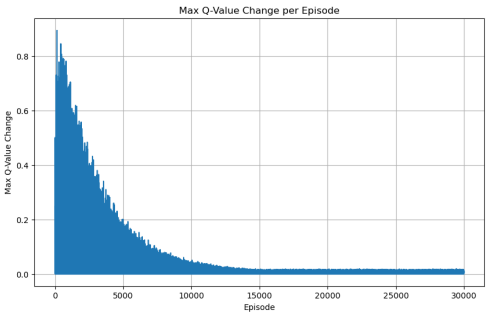

Fig. 9. SARSA-Fixed Q Change per Iteration

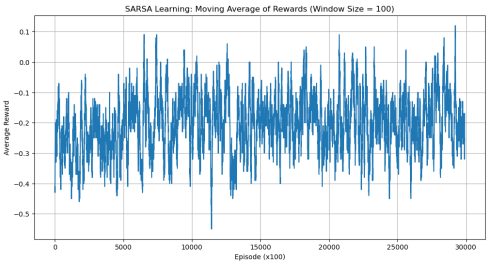

Fig. 10. SARSA-Fixed MA Rewards over Iteration

Epsilon=0.6; Mean Reward: -0.07855; Std: 0.95714

Epsilon=0.7; Mean Reward: -0.08874; Std: 0.95900

Epsilon=0.8; Mean Reward: -0.09054; Std: 0.95676

Epsilon=0.9; Mean Reward: -0.09513; Std: 0.95842

$\epsilon = 0.4$ seems to be the optimal, so I plot figures based on that strategy to represent fixed epsilon strategy. The mean rewards of fixed strategy is mostly not improving compared to decay strategy.

The convergence shown in Fig.9 does not differ with decay strategy significantly (maybe converges slower), but the MA plot shows a more horizontal pattern, which might result from constant exploration rate, spending considerate time to explore new space instead of digging deeper. Correspondingly, the cumulative rewards from Fig.11 is nearly a straight line.

The heatplot also suggests that SARSA's outcomes are highly affected by randomness compared to VI ans PI.

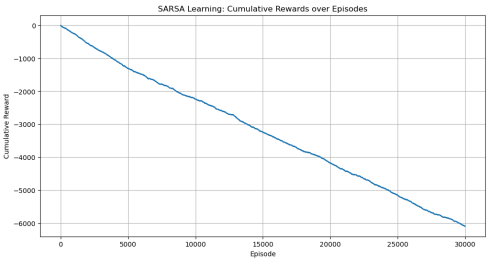

Fig. 11. SARSA-Fixed Cummulative Rewards over Iteration

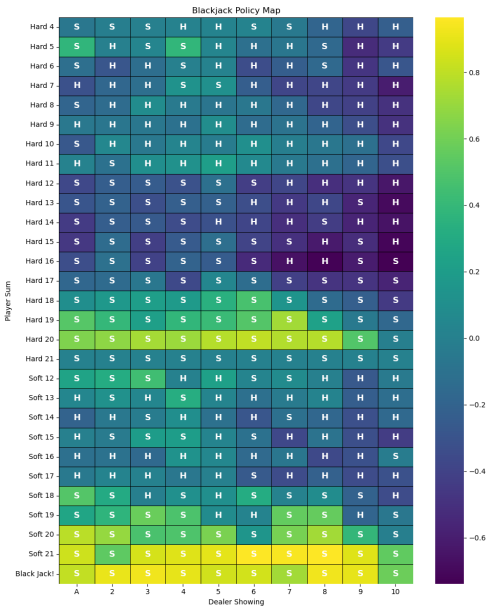

Fig. 12. Blackjack Policy Heatmap(SARSA-Fixed)

IV. EXPERIMENTS ON CART POLE ENVIRONMENT

A. Environment Introduction

CartPole is a classic physics-based control problem. The goal is to balance a pole on top of a cart that moves along a frictionless track. The state of the environment is represented by a vector of four continuous values: the cart's position, the cart's velocity, the pole's angle, and the pole's angular velocity at its tip. The agent can take one of two discrete actions: push the cart to the left or to the right. A reward of +1 is given for every timestep that the pole remains upright. An episode terminates if the pole's angle exceeds a certain threshold from vertical or the cart moves too far from the center.

The CartPole environment is crucial for this report because it represents a continuous and deterministic MDP. Its primary challenge and analytical value stem from its continuous state space, which makes it directly incompatible with tabular methods like Value Iteration (VI) and Policy Iteration (PI) without modification. This necessitates the application of state discretization, a key technique this report is tasked with analyzing. Consequently, CartPole serves as an excellent case for studying the impact of discretization, allowing for an analysis of the trade-offs between the resolution of the discretized grid, computational efficiency, and the accuracy of the resulting policy. Its deterministic nature provides a clear contrast to the stochasticity of Blackjack, enabling a focused study on how these different environmental properties affect algorithm performance.

B. Experiments on Value Iteration

Because Cartpole is a continuous environment, I need to discretize it to apply these algorithms. I'll use three levels of precision, discretized by tuning $threshold_bins$, named as Rough, Medium and Precise.

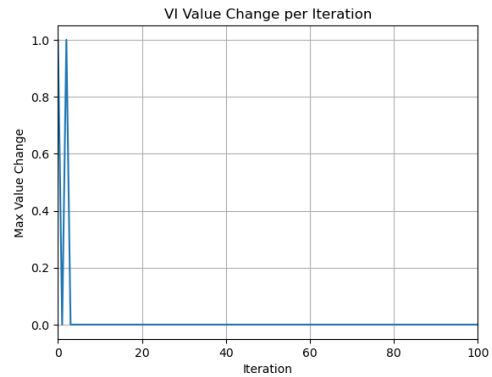

Fig. 13. VI-Rough Value Change per Iteration

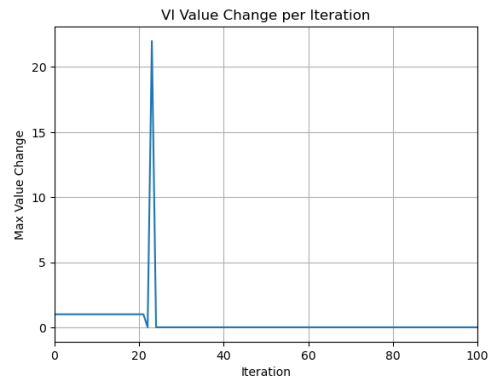

Fig. 14. VI-Medium Value Change per Iteration

The convergence rate is slowed down while I increasing precision. It's easy to see that coming because the search space is broadened when applying more precise discretizing strategy. When the number of bins increases, the time it takes to converge exceeds that it takes in Blackjack case.

The mean rewards using Rough strategy is 9.42; The mean rewards using Medium strategy is 253.15; The mean rewards using Precise strategy is 28.82

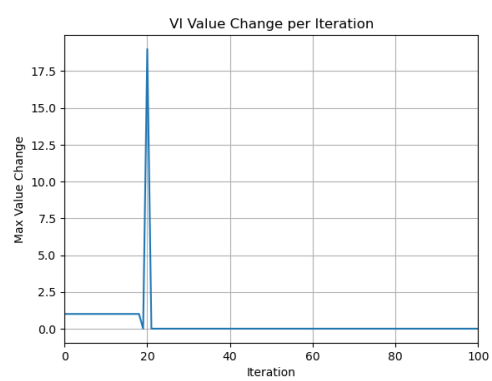

Fig. 15. VI-Precise Value Change per Iteration

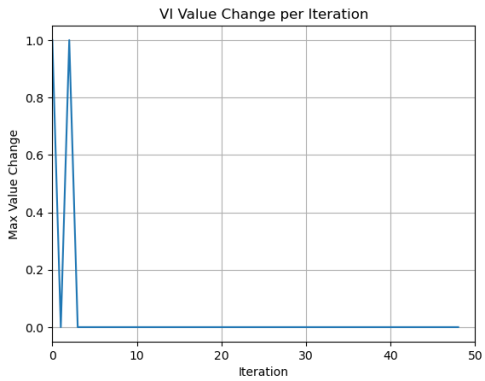

Fig. 16. PI-Rough Value Change per Iteration

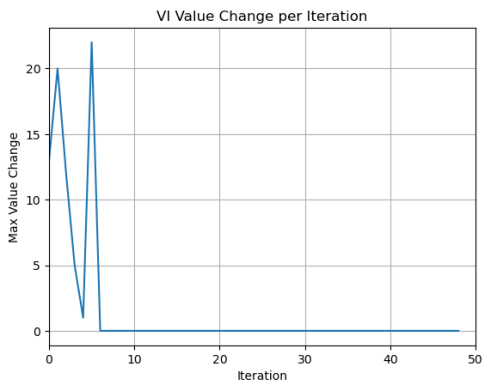

Fig. 17. PI-Medium Value Change per Iteration

C. Experiments on Policy Iteration

The figures show similar tendency those of VI show. When precision or complexity of the states increases, the convergence time increases.

The mean rewards using Rough strategy is 106.346; The mean rewards using Medium strategy is 138.322; The mean rewards using Precise strategy is 132.255

D. Experiments on SARSA

The MA and cumulative plots are basically showing similar information, so I'll only show MA instead of cumulative plots.

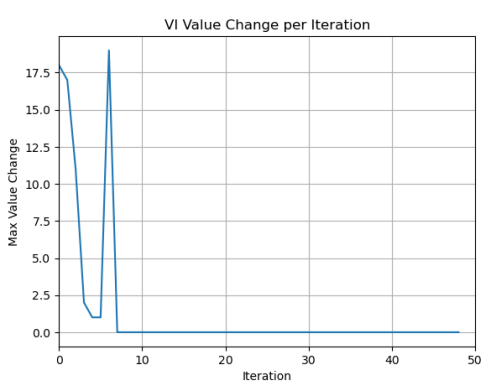

Fig. 18. PI-Precise Value Change per Iteration

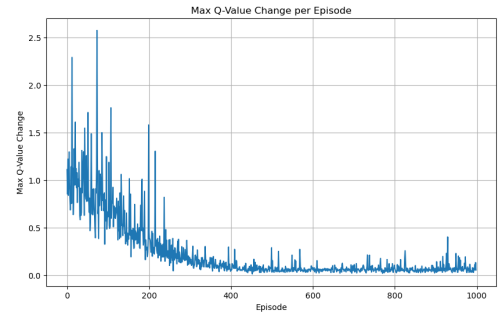

Fig. 19. SARSA-Decay Q Change per Iteration(Precise)

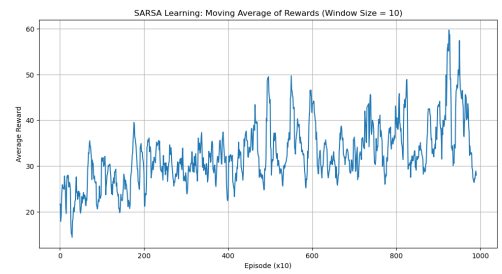

Fig. 20. SARSA-Decay MA Rewards over Iteration(Precise)

1) SARSA using Decaying Epsilon: The mean rewards using Rough strategy is 9.2; The mean rewards using Medium strategy is 255.61; The mean rewards using Precise strategy is 30.17

The performance of SARSA with a decaying epsilon is highly sensitive to the level of state discretization. The mean rewards using Rough, Medium, and Precise strategies were 9.2, 255.61, and 30.17, respectively. Notably, the Medium discretization achieved the best performance, significantly outperforming the other two. The Precise strategy, despite its higher resolution, performed poorly. This is likely because a finer discretization leads to a much larger state space, making it significantly harder for the model-free SARSA agent to explore effectively and converge to an optimal policy within a limited number of episodes.

The convergence plot for the Precise strategy, shown in Fig.19, exhibits the characteristic behavior of a model-free learning method: high-frequency oscillations in the Q-value change that slowly trend toward zero over many episodes. This contrasts sharply with the rapid and deterministic convergence of Value Iteration and Policy Iteration. The Moving Average (MA) rewards plot (Fig.20) and the cumulative rewards plot (Fig.21) for the Precise setting further confirm its suboptimal performance; the agent's rewards improve initially but fail to reach a high and stable level, consistent with the low mean reward.

2) SARSA using Fixed Epsilon: I tried 9 ϵ value from 0.1 to 0.9 with step size 0.1. And the outcomes are:

Epsilon=0.1; Mean Reward: 83.264; Std: 24.562
Epsilon=0.2; Mean Reward: 302.389; Std: 111.553
Epsilon=0.3; Mean Reward: 172.497; Std: 51.656
Epsilon=0.4; Mean Reward: 138.254; Std: 36.491

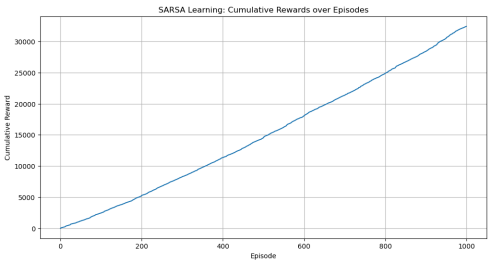

Fig. 21. SARSA-Decay Cummulative Rewards over Iteration(Precise)

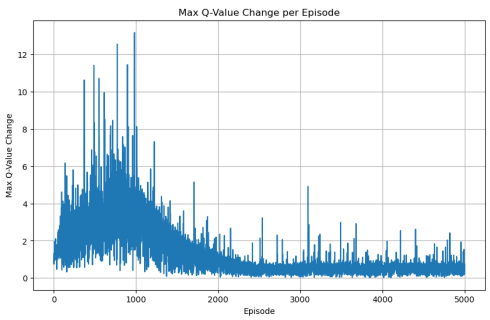

Fig. 22. SARSA-Fixed Q Change per Iteration(Precise)

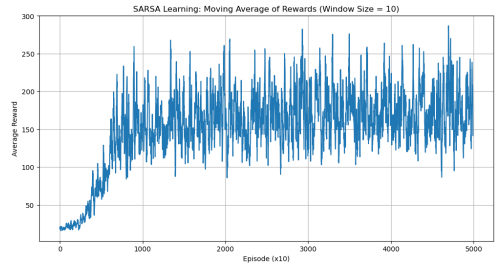

Fig. 23. SARSA-Fixed MA Rewards over Iteration(Precise)

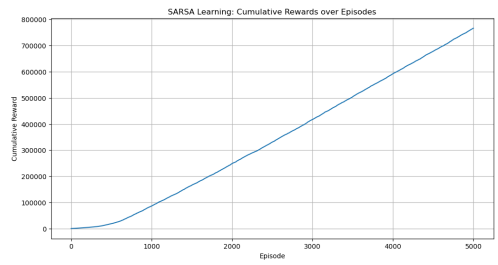

Fig. 24. SARSA-Fixed Cummulative Rewards over Iteration(Precise)

Epsilon=0.5; Mean Reward: 287.843; Std: 118.945
Epsilon=0.6; Mean Reward: 243.653; Std: 94.422
Epsilon=0.7; Mean Reward: 186.668; Std: 64.650
Epsilon=0.8; Mean Reward: 165.304; Std: 51.886
Epsilon=0.9; Mean Reward: 155.816; Std: 49.885

$\epsilon = 0.2$ yielding the best mean reward of 302.389. This optimal result with a fixed epsilon surpassed the best performance achieved with a decaying epsilon strategy.

The results for different discretization levels were also recorded. Similar to the decaying epsilon case, the Medium discretization proved most effective. The convergence plot for the Precise strategy (Fig. 22) shows a pattern of oscillating Q-values similar to that of the decaying epsilon strategy (Fig.19). However, the MA rewards plot for the Precise strategy (Fig.23) suggests slightly more stable learning compared to its decaying epsilon counterpart (Fig.20), although the overall reward remains low. The cumulative rewards plot (Fig.24) reflects this steady, albeit slow, accumulation of rewards. Overall, while a well-tuned fixed epsilon can outperform a decaying one, both SARSA approaches are highly dependent on the chosen discretization level, with overly fine grids hindering effective policy discovery.

The mean rewards using Rough strategy is 9.2; The mean rewards using Medium strategy is 255.61; The mean rewards using Precise strategy is 30.17

V. CONCLUSIONS

This report conducted a comparative analysis of Value Iteration, Policy Iteration, and SARSA on the discrete stochastic Blackjack environment and the continuous deterministic CartPole environment. The experimental results reasonably confirmed our initial assumptions and offered some valuable observations on the real-world application of these algorithms.

1. Model-based methods were found to perform better and be more optimal as anticipated. VI and PI for both Blackjack and CartPole both converged toward better policies in less number of iterations than SARSA needed. This shows the central advantage of a world model in allowing efficient planning.

2. Policy Iteration converged faster than Value Iteration in terms of iterations in all cases, particularly in Blackjack where it was converging in roughly 5 iterations to VI's 12. This supports the hypothesis that optimizing directly on the policy space can be easier if the number of optimal policies is limited.

3. Discretization of the state space was another major contributor for CartPole. As can be anticipated, the "Medium" discretization method fared best on all agents. The "Precise" method, by creating too large of a state space, had a very negative effect on performance, especially for the model-free SARSA agent (average reward of 30.17 vs. 255.61 for the Medium grid with epsilon decay), quintessentially illustrating the curse of dimensionality.

4. The optimal exploration policy was indeed task-dependent. For balancing task of CartPole, a tuned fixed epsilon (ϵ=0.2) achieved the highest mean reward (302.389), which outperformed the top decaying epsilon policy (255.61). This confirms our assumption that continuous exploration is beneficial to such dynamic control tasks.

The greatest challenge was the tradeoff between representation accuracy and computational tractability, specifically discretizing the CartPole state space. Future work could involve using function approximation methods like DQN to natively more support continuous state spaces and maybe even do even better without discretization.